



SOLUTION BRIEF

# Agentic Coding - Shared Memory Layer

---

# One shared memory for a team of agents.

**Bottom line.** mem9 gives a team of AI coding agents **one shared, queryable memory** on TiDB Cloud: agents read before they build and write back when done, so none starts cold or duplicates work - across sessions, tools, and repos. That shared memory is the product. What makes it work is the engine underneath: because mem9 runs on TiDB, not a bolt-on vector store, the same memory also answers the one question similarity search cannot - the transitive blast radius of a change. Isolated per team, and every screenshot in this brief is a live read or write against TiDB Cloud.

3

coding agents,  
different vendors

4

components built  
to parity

0

duplicates, re-briefs,  
or cold starts

1

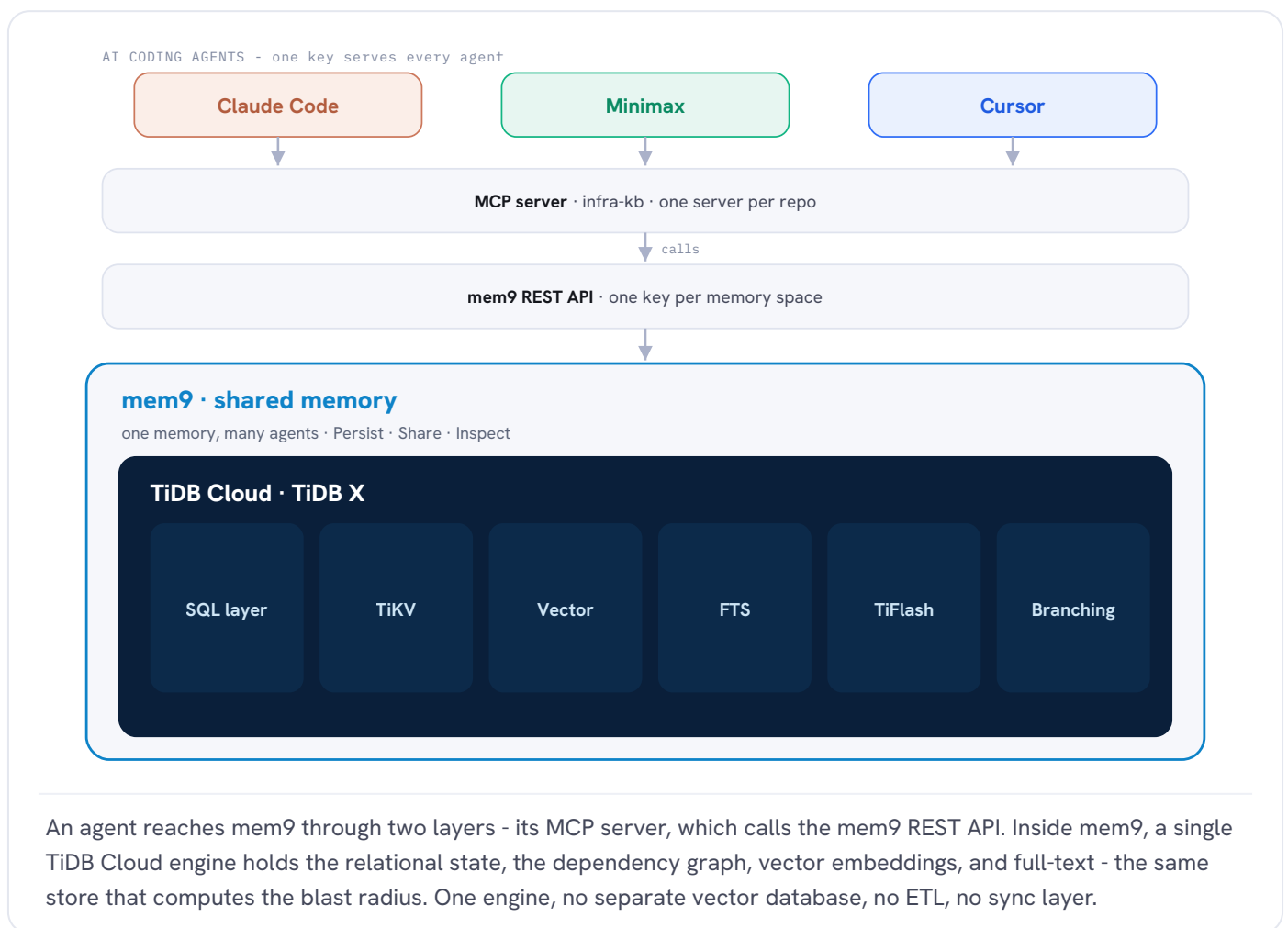
shared memory  
on TiDB Cloud

| THE PROBLEM                                                                                    | HOW MEM9 SOLVES IT                                                                                                                                                                                                                                   | SHOWN IN                              |
|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Each team's knowledge must stay private - no team reading another's.                           | One <b>TiDB Cloud cluster per team</b> (a mem9 space).<br>A team's key reaches only its own memory; isolation is enforced by the platform, not application code.                                                                                     | <a href="#">§ The model</a>           |
| The same integration should work for every tool and every repo - no custom glue.               | One MCP pattern: each repo is a <b>named server scoped to its <code>appId</code> namespace</b> , and the agent routes to the right one. The same wiring serves Claude Code, Minimax, and Cursor; across teams, separate keys mean separate clusters. | <a href="#">§ The model</a>           |
| A single task often spans repos - create a resource in one, the dependent resource in another. | One agent session writes to <b>each repo's namespace</b> and links the records by <code>account_ref</code> - one session, two repos, no manual hand-off.                                                                                             | <a href="#">§ One task, two repos</a> |
| Some teams require a self-hosted deployment.                                                   | <b>Supported.</b> Vector and graph run self-hosted; full-text search is TiDB Cloud only today.                                                                                                                                                       | <a href="#">§ Deployment</a>          |

Demonstrated live with three different coding agents - Claude Code, Minimax, and Cursor - against a single TiDB Cloud cluster. The tools are interchangeable; the shared memory is the point.

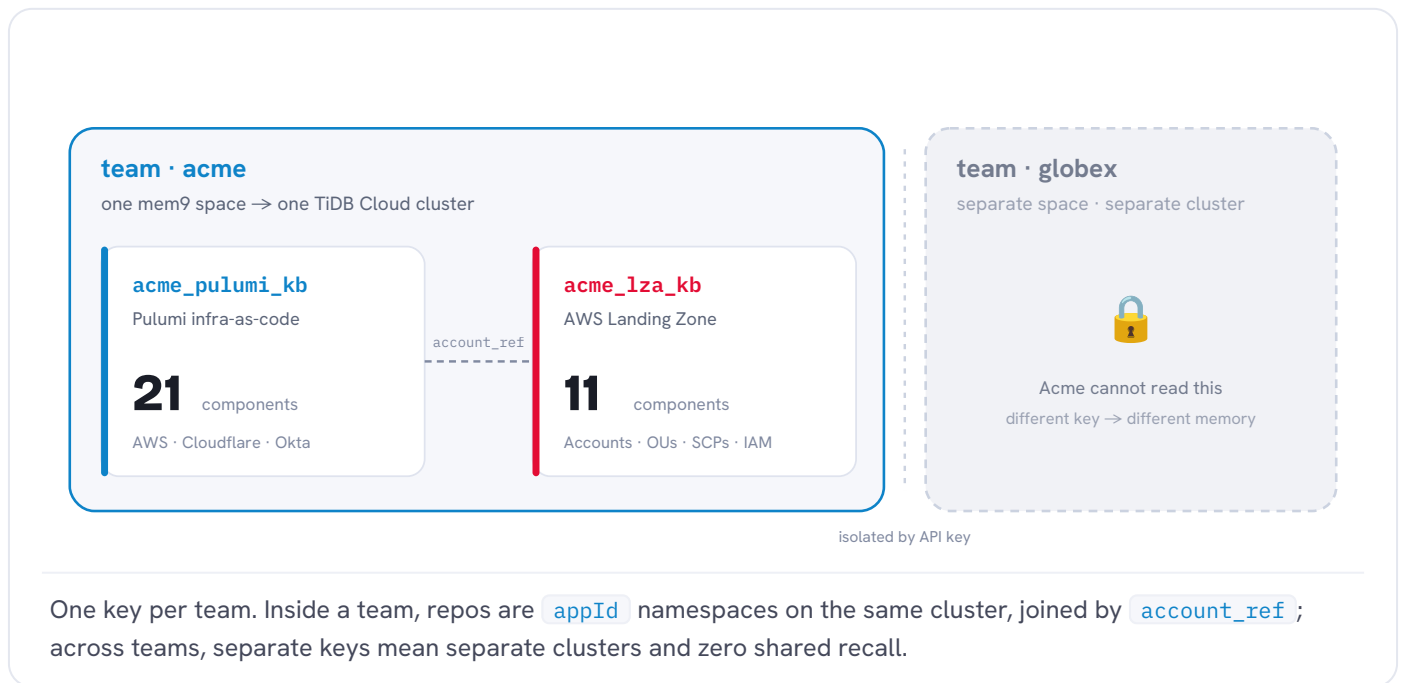
# mem9 is the memory. TiDB is the engine.

mem9 is the agent-facing product - **one memory, many agents**: a shared, persistent layer every coding agent reads and writes through a single key, with hybrid recall and a human-inspectable dashboard. An agent reaches it through two layers - its **MCP server**, which calls the **mem9 REST API**. And mem9 is TiDB Cloud at its core: the SQL layer, TiKV, vector and full-text search, TiFlash, and branching that store and serve every memory all run inside it - one engine, no separate vector database or sync layer.



# Team = cluster. Repo = namespace.

mem9 maps onto how a team already works. A **team** is a mem9 **space** - one key, one isolated TiDB Cloud cluster. A **repo** is an `appId` namespace inside it, so repos share a cluster but keep separate recall scopes. Another team is another cluster, reachable only with its own key.



Each team's space sits on its own managed TiDB Cloud cluster. Agents never handle raw database credentials - the mem9 API key is the only secret.

# One task, two repos.

A single task often spans repos - create a new AWS account in the LZA repo *and* the S3 bucket for it in the Pulumi repo. One agent, given one plain-language request, writes the account to `acme_lza_kb` and the bucket to `acme_pulumi_kb` - two separate namespaces - and links them with `account_ref`. No second session, no manual hand-off.

mem9

Infra Knowledge Base

acme - pulumi + lza

TIDB Cloud

Reset

Live

Cross-repo

Dependencies

Search

Config

## One task. Two repos. One memory.

A single agent session provisions a new AWS account in the **LZA** repo and an S3 bucket in the **Pulumi** repo deployed into it - reading and writing across both `appId` namespaces in one MCP session, joined by `account_ref`.

Run cross-repo task

dev types: "Create a new data-platform AWS account in LZA and an exports bucket for it in Pulumi."

cross-repo task complete

ACTIVITY · BOTH REPOS

claude-code

WRITE

pulumi

Created acme-prod-data-platform-exports, an S3 bucket deployed into the new data-platform account.

claude-code

WRITE

lza

Created acme-lza-account-data-platform, a new AWS account in the workloads OU.

ACCOUNT\_REF JOIN · LZA ACCOUNT ↔ PULUMI RESOURCES

lza

account-data-platform

NEW

pulumi

data-platform-exports

account\_ref=platform

lza

account-prod

account\_ref=prod

pulumi

analytics-db

ROS

data-exports

S3

static-assets

S3

assets-dns

Cloudflare

admin-dns

Cloudflare

admin-ss0

Okta

+6 more

lza

account-sandbox

account\_ref=sandbox

pulumi

analytics-db

ROS

data-exports

S3

DASHBOARD · CROSS-REPO

Left: one activity feed shows the same agent writing to **both** repos. Right: the `account_ref` join - the new **data-platform** account and its Pulumi exports bucket, created together and linked.

**Why it matters.** The agent doesn't need to be told which repo owns what. It routes each write to the right namespace and records the cross-repo link, so the next session - in either repo - already knows the account and its bucket belong together.

# Three tools, one memory, warm hand-offs.

Production is complete; staging has drifted by four components. Three different agents bring it back to parity - the developer only states the goal. Every agent's rules file ( [CLAUDE.md](#) / [AGENTS.md](#) / [.cursorrules](#) ) makes the same first move non-negotiable: **query mem9 before writing anything**. So each step below runs the same shape - recall, compose, write back. Each **write** records the component and its dependency edges in the shared memory - the knowledge base the next agent reads - alongside the code the agent generates in the repo.

The screenshot displays the mem9 Infra Knowledge Base interface. At the top, there's a header with 'mem9 Infra Knowledge Base', a breadcrumb 'acme - pulumi + lza', and a 'TiDB Cloud' button. Below the header is a navigation bar with 'Live', 'Cross-repo', 'Dependencies', 'Search', and 'Config'. The main content area is titled 'Three tools, one memory, live.' and includes a sub-header: 'Every query and write from Claude Code, Minimax, and Cursor lands in the same mem9 space. Watch the staging gap close as each tool reads the shared log and writes back.'

The interface is divided into two main sections. On the left is the 'ACTIVITY' panel, which currently shows 'No activity yet. Launch a CLI and ask it to query the knowledge base.' On the right is the 'KNOWLEDGE GRAPH' panel, which displays a network of nodes and edges. The nodes are labeled with icons and names: 'analytics-db', 'data-exports', 'static-assets', 'assets-dns', 'portal-svc', 'data-kms', 'ScpPolicy', and 'assets-dns'. The edges represent dependencies between these components. Below the graph is a 'STAGING PARITY' section, which shows a list of components and their status in staging. The status is indicated by a checkmark (green) for 'in staging' or an 'X' (orange) for 'missing'. The components are: 'analytics-db' (in staging), 'data-exports' (in staging), 'static-assets' (missing), and 'assets-dns' (missing). The 'STAGING PARITY' section also includes a summary '4 missing in staging'.

At the bottom of the interface, there's a 'STARTING POINT' section with the text: 'Feed empty; the staging-parity strip shows 4 missing.'

## STEP 1

# Claude Code

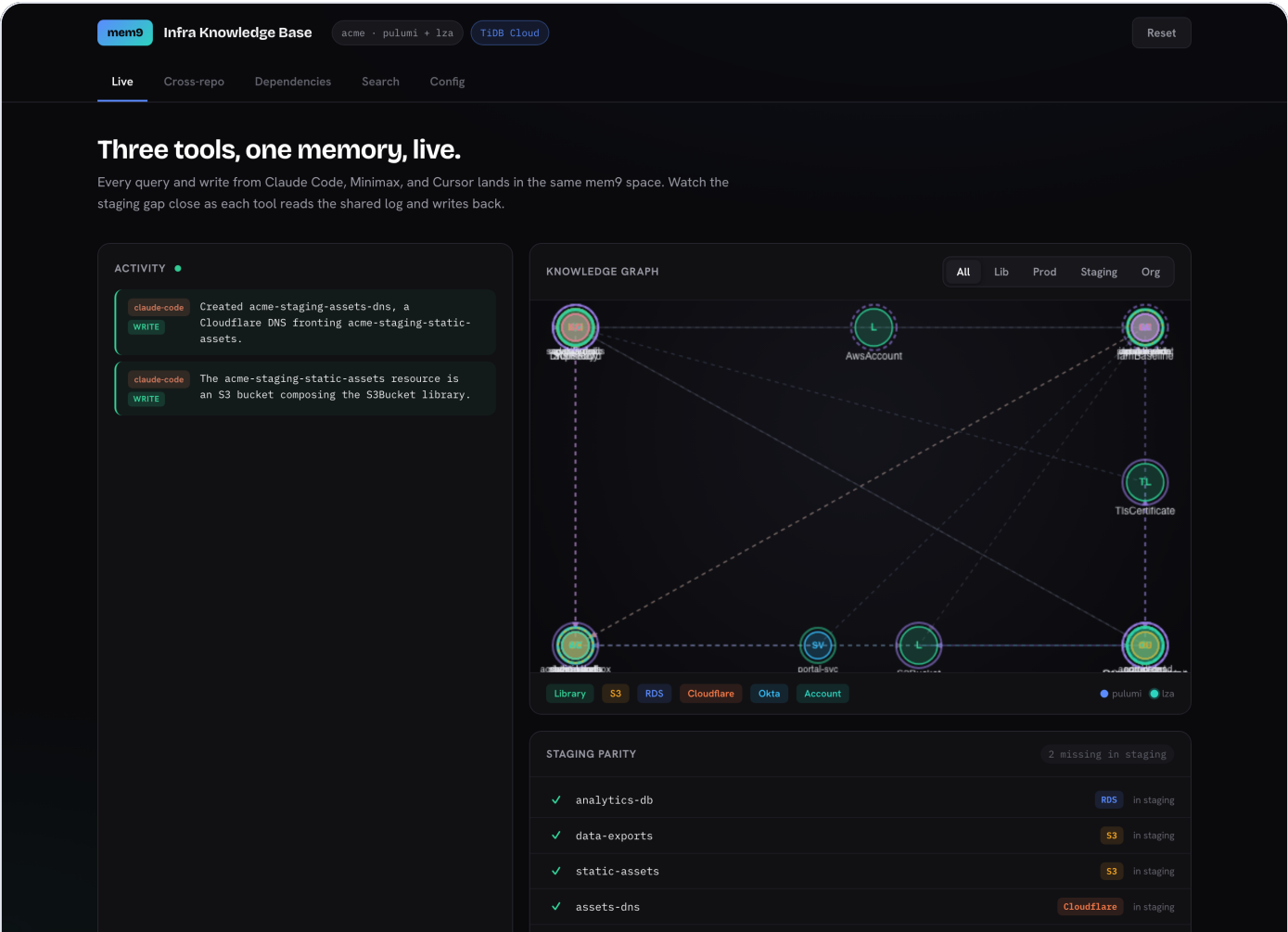
dev types "Bring staging up to parity with production - start with the static-assets bucket and its CDN."

QUERY-FIRST - the rules file requires a recall before any write



recall the gap → compose the S3 + DNS libraries → write both components back, with the dns-fronts-bucket edge.

**Result.** Staging gains the **static-assets** bucket and its **CDN**. **Gap 4 → 2.**



RESULT · CLAUDE CODE

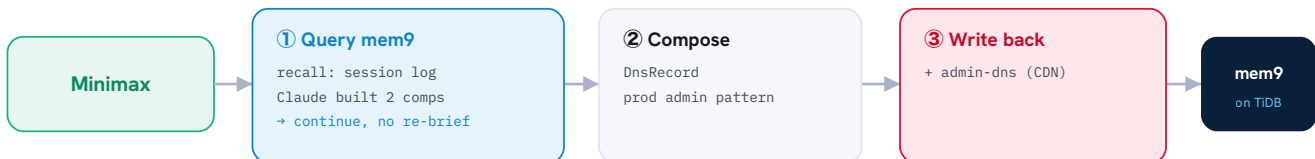
Both writes land in the shared feed; the parity strip drops to 2 missing.

## STEP 2

## Minimax

dev types "Keep bringing staging to parity with prod - it still needs the admin-portal DNS."

QUERY-FIRST - Minimax opens cold and reads the shared log before it builds



recall the shared session log (what Claude just built) → compose the admin DnsRecord → write it back. No re-briefing.

**Result.** The admin-portal DNS lands - Minimax never had to be re-briefed. **Gap 2 → 1.**



mem9

Infra Knowledge Base

acme · pulumi + lza

TiDB Cloud

Reset

Live

Cross-repo

Dependencies

Search

Config

## Three tools, one memory, live.

Every query and write from Claude Code, Minimax, and Cursor lands in the same mem9 space. Watch the staging gap close as each tool reads the shared log and writes back.

ACTIVITY

minimax

WRITE

Created acme-staging-admin-dns, a Cloudflare DNS matching the production admin pattern.

minimax

QUERY

The user is continuing staging work.

claude-code

WRITE

Created acme-staging-assets-dns, a Cloudflare DNS fronting acme-staging-static-assets.

claude-code

WRITE

The acme-staging-static-assets resource is an S3 bucket composing the S3Bucket library.

KNOWLEDGE GRAPH

All

Lib

Prod

Staging

Org

Library

S3

RDS

Cloudflare

Okta

Account

pulumi

lza

STAGING PARITY

1 missing in staging

|                 |            |            |
|-----------------|------------|------------|
| ✓ analytics-db  | RDS        | in staging |
| ✓ data-exports  | S3         | in staging |
| ✓ static-assets | S3         | in staging |
| ✓ assets-dns    | Cloudflare | in staging |

RESULT · MINIMAX

Its query ("continuing staging work") and write join the same timeline as Claude Code's. 1 missing.

**Interchangeable by construction.** This seat ran OpenAI's Codex earlier in the project and was swapped to MiniMax-M3 mid-stream - with zero changes to mem9, the schema, or the MCP wiring. Any agent that speaks MCP reads the same session log and picks up the same warm context.

### STEP 3 Cursor

dev types "Finish bringing staging to parity - add the admin SSO app."

QUERY-FIRST - Cursor traces the dependency chain before it builds



hybrid-search the prod SSO app → trace its depends\_on / redirects\_to chain → compose and write the matching staging SSO app.

**Result.** The SSO app's redirect URI points at the admin DNS; staging reaches **parity**. **Gap 1 → 0.**

mem9

Infra Knowledge Base

acme · pulumi + lza

TiDB Cloud

Reset

LiveCross-repoDependenciesSearchConfig

### Three tools, one memory, live.

Every query and write from Claude Code, Minimax, and Cursor lands in the same mem9 space. Watch the staging gap close as each tool reads the shared log and writes back.

ACTIVITY

cursor

WRITE

Created acme-staging-admin-ss0, an Okta SSO app whose redirect URI points at acme-staging-admin-dns.

cursor

WRITE

Staging is now at parity with production.

minimax

WRITE

Created acme-staging-admin-dns, a Cloudflare DNS matching the production admin pattern.

minimax

QUERY

The user is continuing staging work.

claude-code

WRITE

Created acme-staging-assets-dns, a Cloudflare DNS fronting acme-staging-static-assets.

claude-code

WRITE

The acme-staging-static-assets resource is an S3 bucket composing the S3Bucket library.

KNOWLEDGE GRAPH

AllLibProdStagingOrg

LibraryS3RDSCloudflareOktaAccount

STAGING PARITY

at parity

✓ analytics-db

RDS

In staging

✓ data-exports

S3

In staging

✓ static-assets

S3

In staging

✓ assets-dns

Cloudflare

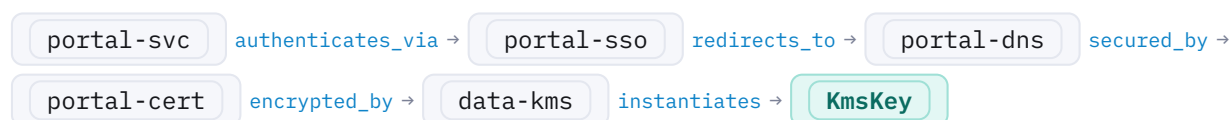
In staging

RESULT · CURSOR

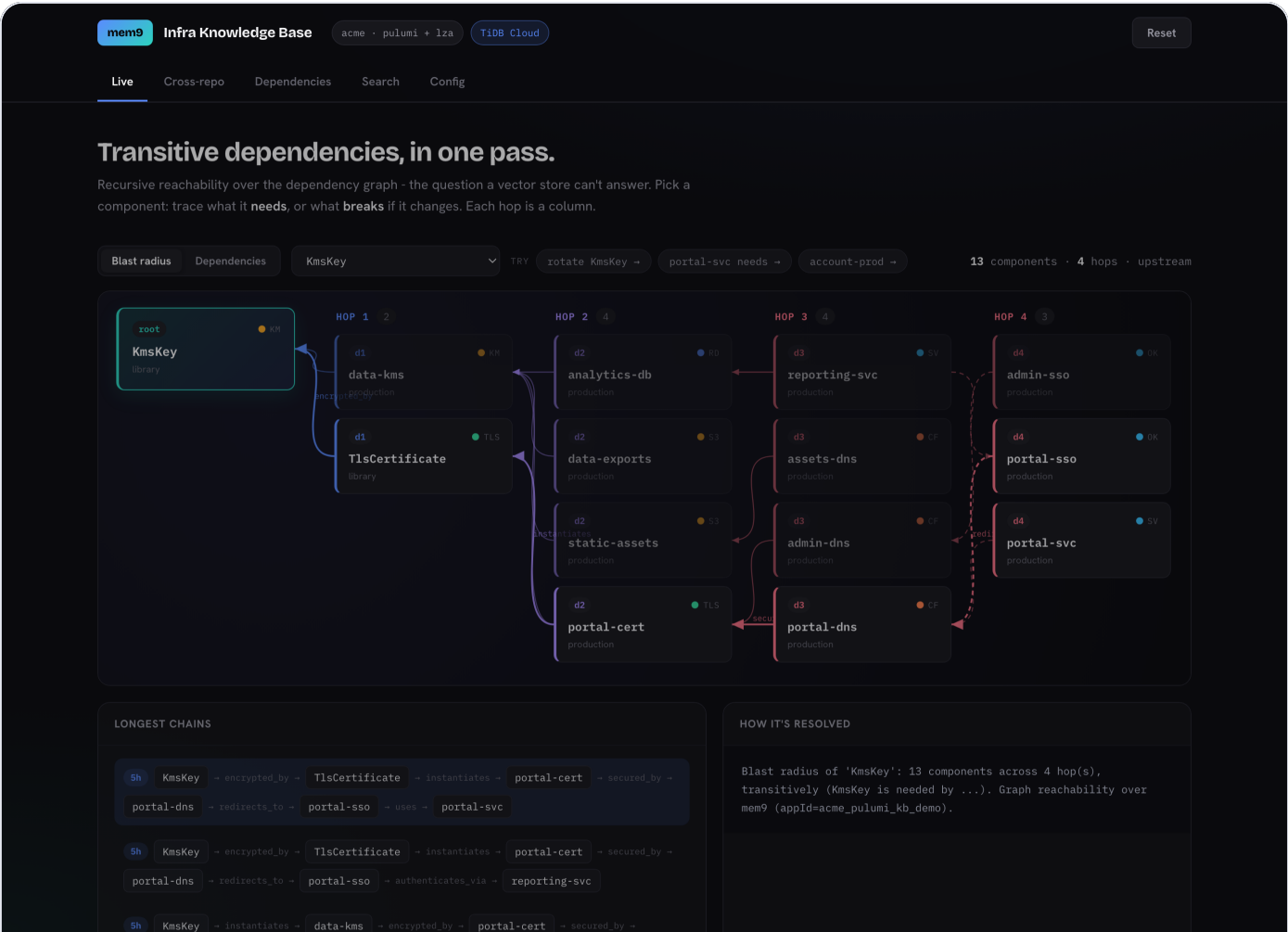
One timeline now spans all three tools; the final write reads "Staging is now at parity with production." **0 missing.**

# The question similarity can't answer.

A vector store finds things that *look* related. It cannot compute a transitive closure - "what, exactly, breaks if I change this?" That is graph reachability. mem9 records every `depends_on` and composition edge - **the agents write the edges as they compose** (the same write that records a component records its links), so the graph stays current with the code. One recursive traversal then walks the chain to any depth.



Run that in reverse from `KmsKey` - "what breaks if we rotate it?" - and the answer is **13 components across four depths**. No flat search surfaces this; only walking the graph does.



[DASHBOARD](#) · [DEPENDENCIES](#)

"What breaks if we rotate **KmsKey**?" Each hop is a column; 13 components fan across four depths and funnel onto the anchored foundation. The Trace panel reads each chain as a sentence.

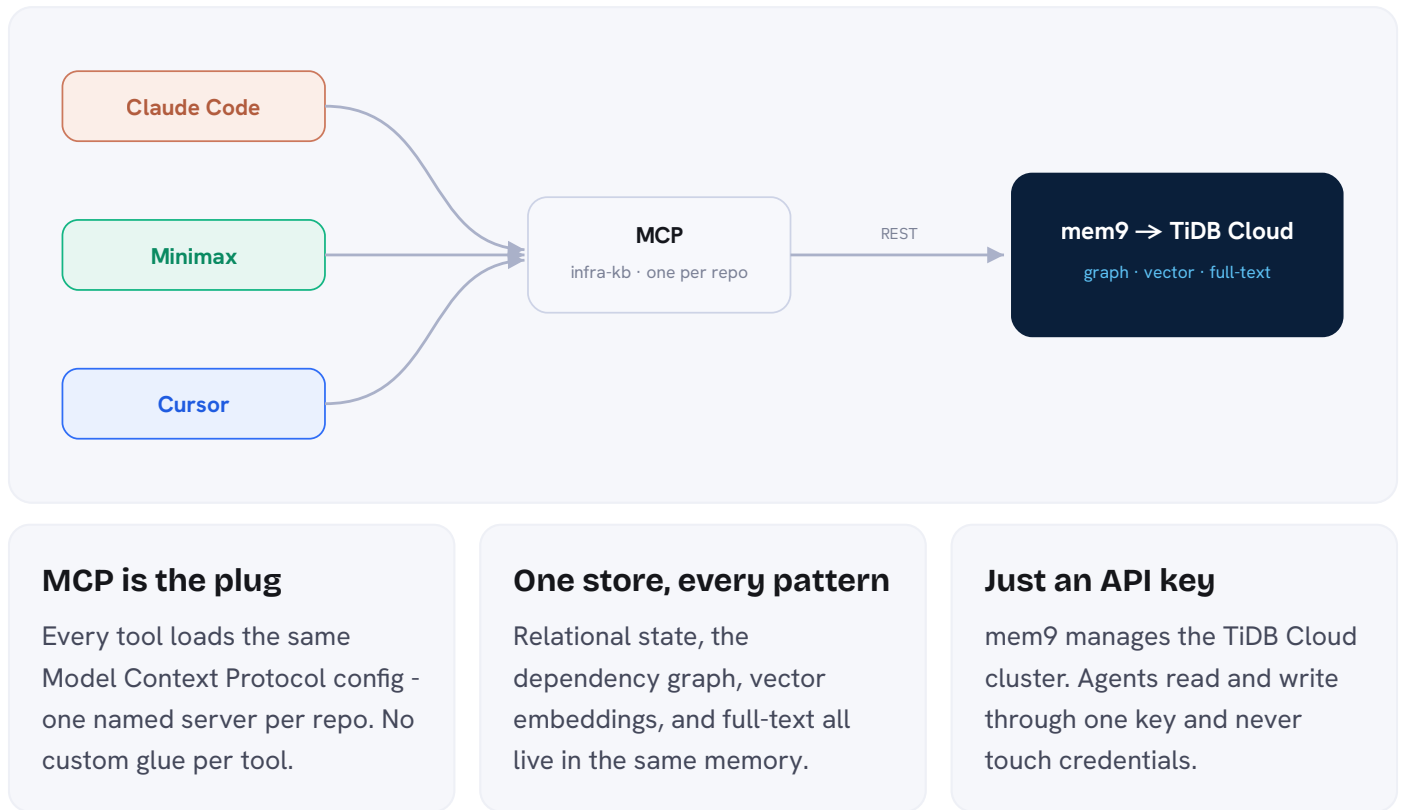
## One engine, not four

| APPROACH              | GRAPH REACHABILITY        | SEMANTIC RECALL    | ATOMIC AGENT STATE        |
|-----------------------|---------------------------|--------------------|---------------------------|
| S3 / filesystem       | no traversal              | no                 | no transactions           |
| Vector DB only        | similarity ≠ reachability | yes                | no                        |
| Four separate systems | partial, with ETL         | yes                | no cross-system atomicity |
| mem9 → TiDB Cloud     | recursive traversal       | vector + full-text | one transaction           |

Graph, vector embeddings, and full-text live in the same store - the same recall that answers "what encrypts production data" also computes the blast radius above. No separate vector DB, no ETL; every tool's write lands atomically in one transaction.

S3 is the filing cabinet for artifacts. mem9 is the brain agents query.

# One store. One plug. One key.



## The standing instruction

The developer types a normal request. The prescriptive part lives in each tool's rules file ( [CLAUDE.md](#) / [AGENTS.md](#) / [.cursorrules](#) ), so the agent queries first, composes, and writes back on every run with no per-task hand-holding.

mem9

Infra Knowledge Base

acme · pulumi + lza

TiDB Cloud

Reset

Live

Cross-repo

Dependencies

Search

Config

## Behavior is configuration, not code.

The developer just types a normal request - "bring staging to parity." This standing instruction, loaded into every tool's rules file, makes the agent query first, compose, and write back on its own. No per-task hand-holding.

STANDING INSTRUCTION · CLAUDE.MD / AGENTS.MD / .CURSORRULES

Before you write or change any infrastructure - automatically, every task:

1. Query first. query\_knowledge\_base on the infra-kb MCP for the repo you're touching: what exists, how it connects, recent log.
2. Compose. Build from libs/ (S3Bucket, PostgresDatabase, DnsRecord, SsoApplication, KmsKey, ...). Never raw aws.\*/cloudflare.\*/okta.\*.
3. Conventions. acme-<env>-<name> naming; required tags; DnsRecord proxied; SSO redirect -> DnsRecord; set account\_ref (prod/sandbox).
4. Blast radius. Before changing a shared component, trace what depends on it.
5. Write back. After any change, call write\_component to record it - or the next tool, and the next session, start cold.

The human never says these steps. They say what they want; you do this on your own.

MCP SERVERS · ONE ENTRY PER REPO

```
{
  "mcpServers": {
    "infra-kb-pulumi": { "command": "python", "args": ["-m", "src.mcp_server"] },
    "infra-kb-lza": { "command": "python", "args": ["-m", "src.mcp_server"] }
  }
}
```

query\_knowledge\_base · write\_component -> hybrid recall + atomic writes

DASHBOARD · CONFIG

The five-step standing instruction every tool loads, and the one-entry-per-repo MCP wiring that makes it automatic.

## Deployment

- **TiDB Cloud (shown here):** graph + vector + full-text, fully managed; nothing to host.
- **Self-hosted:** available - vector and graph run on-prem. Full-text search is TiDB Cloud only today, so this brief uses Cloud to show hybrid recall end to end.

# From brief to adoption

This brief shows the *what* and the *why* on live infrastructure. The technical deep-dive covers the proof an evaluator needs next:

## Proof it works

The same staging-parity task run **with and without** mem9, plus raw agent traces - prompt, recall, write-back, and the resulting repo diff.

## Security & tenancy

API-key scoping, audit logging, and where isolation is enforced - exactly what one key can read and write inside a team.

## Scale & cost

Read, write, and traversal latency; RU / RCU sizing; and the economics of a cluster per team at fleet scale.

## Correctness over time

How edges are written and validated, how stale memory is detected, and how concurrent or conflicting writes reconcile.

---

mem9 · TiDB Cloud - shared memory for AI coding agents · v3. Live demo: Claude Code · Minimax · Cursor over team acme (repos pulumi + lza). Every figure is a live read/write against TiDB Cloud. Demo source: [github.com/stephenlthorn/mem9-ai-coding](https://github.com/stephenlthorn/mem9-ai-coding) · Apache-2.0.